



COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Hashtables – Definition and Hash Functions

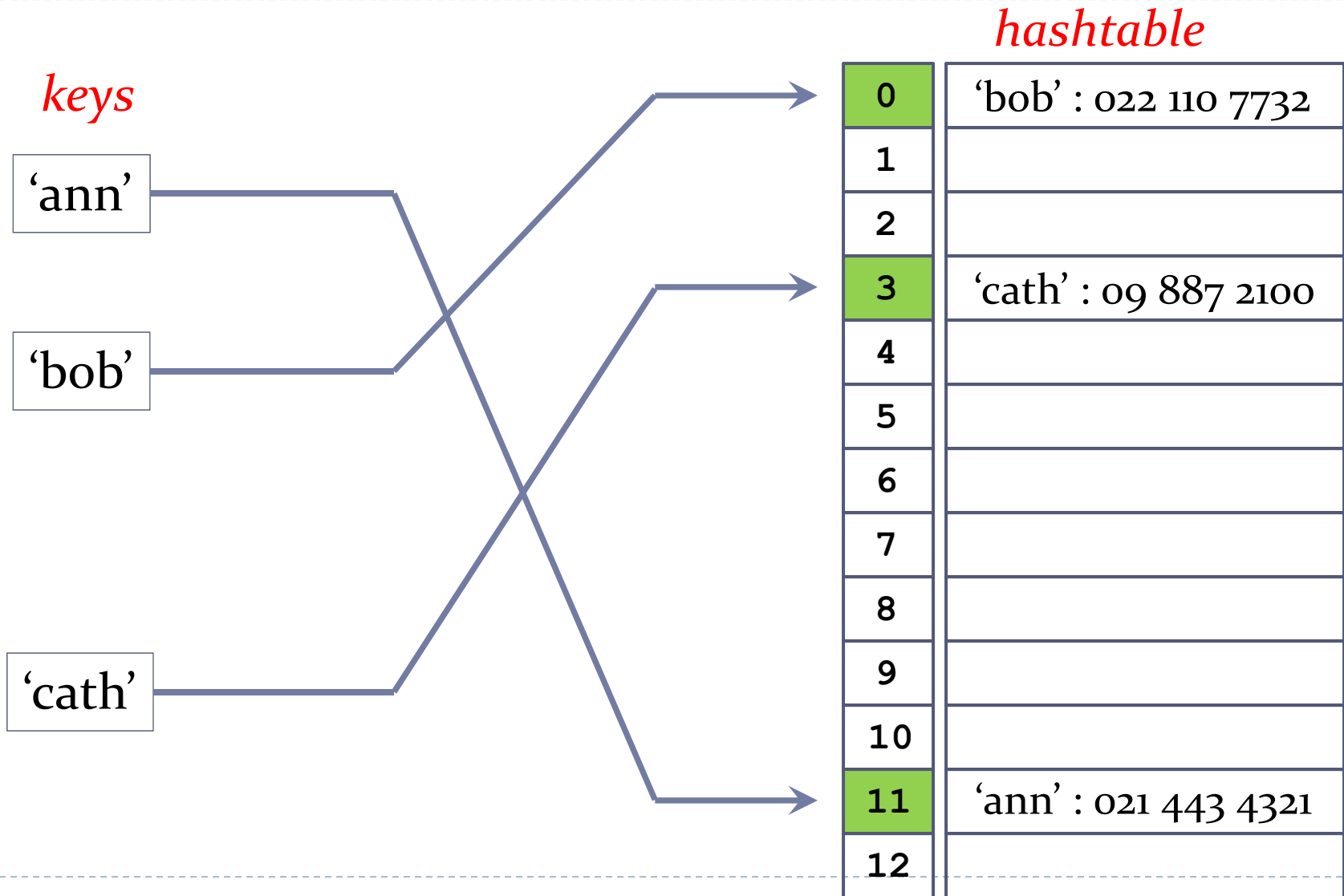


Hashtable definition

- ▶ A *hashtable* is a collection of items stored in a table (the items typically consist of a **key** and a **value**)
 - ▶ for example, the key might be a name, and the value might be a phone number
- ▶ We can think of the table like a normal Python list (with index values that start at 0)
- ▶ The key of each item *maps* to a particular index position in the table, and that is where the value is stored
- ▶ This mapping is performed by a *hash function*
 - ▶ If two keys map to the same index position in the table, then we have a **collision**. We can handle this using either “open addressing” or “separate chaining”.
- ▶ Let’s look at a conceptual view of a hashtable...

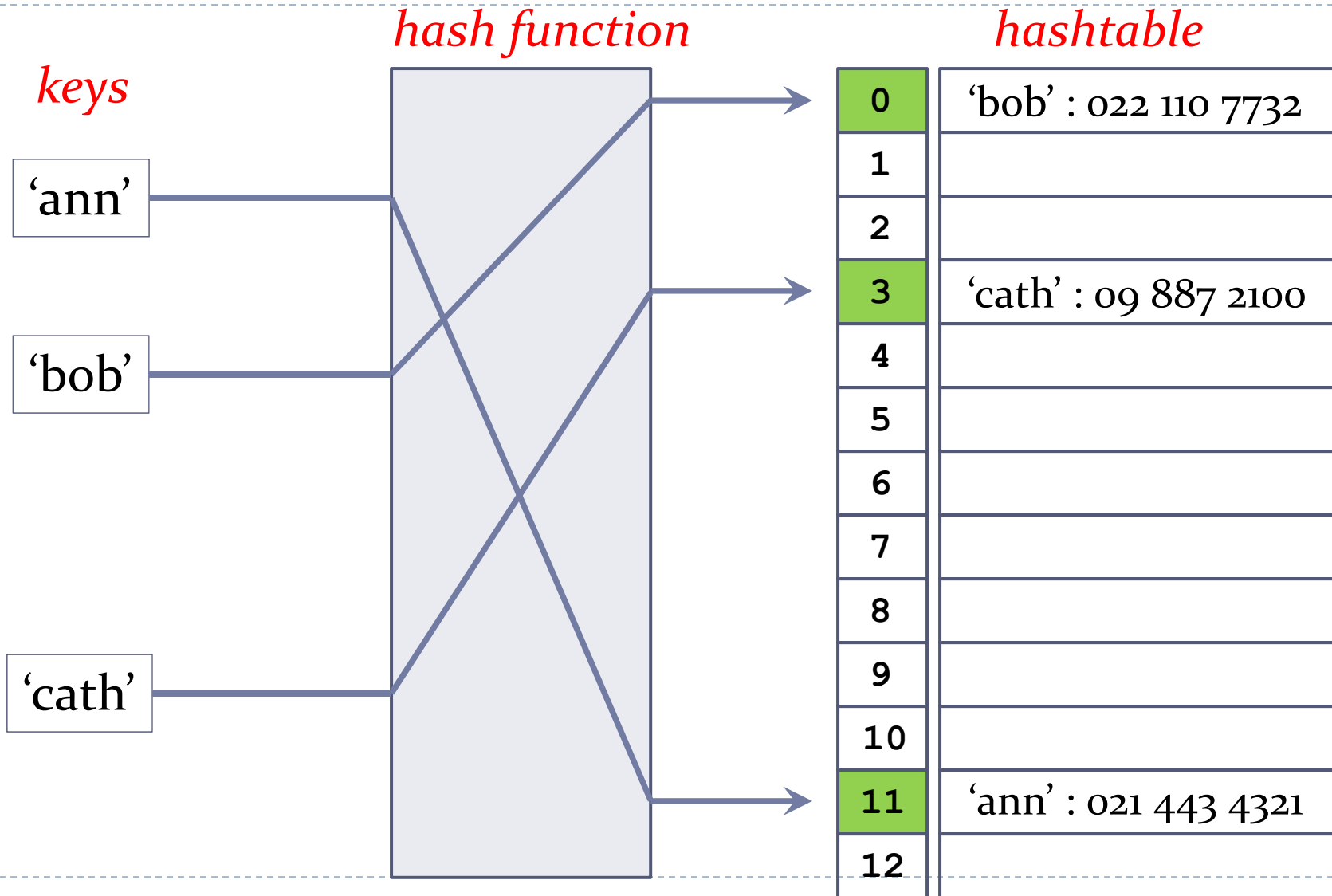


A conceptual view of a hashtable



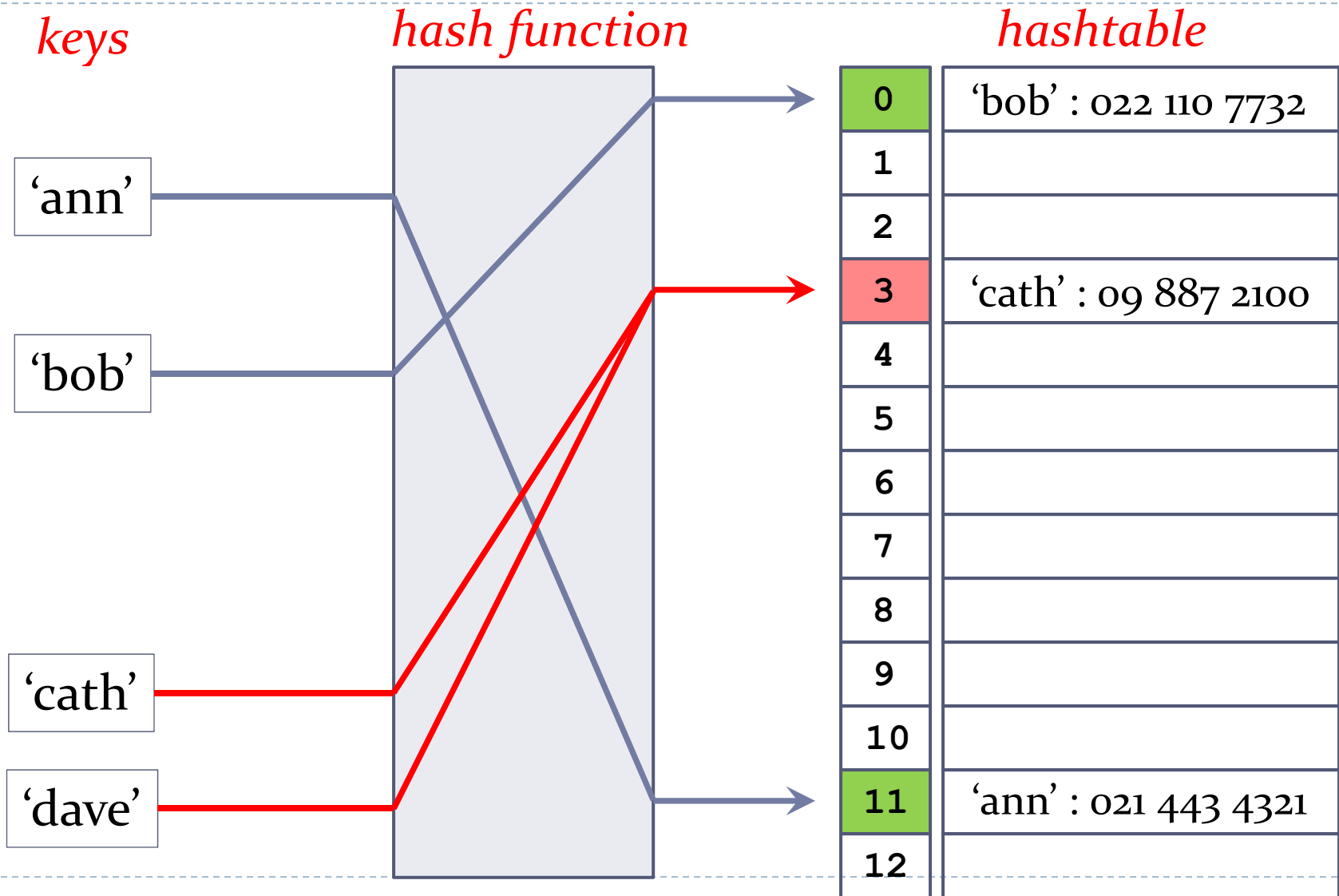


A conceptual view of a hashtable





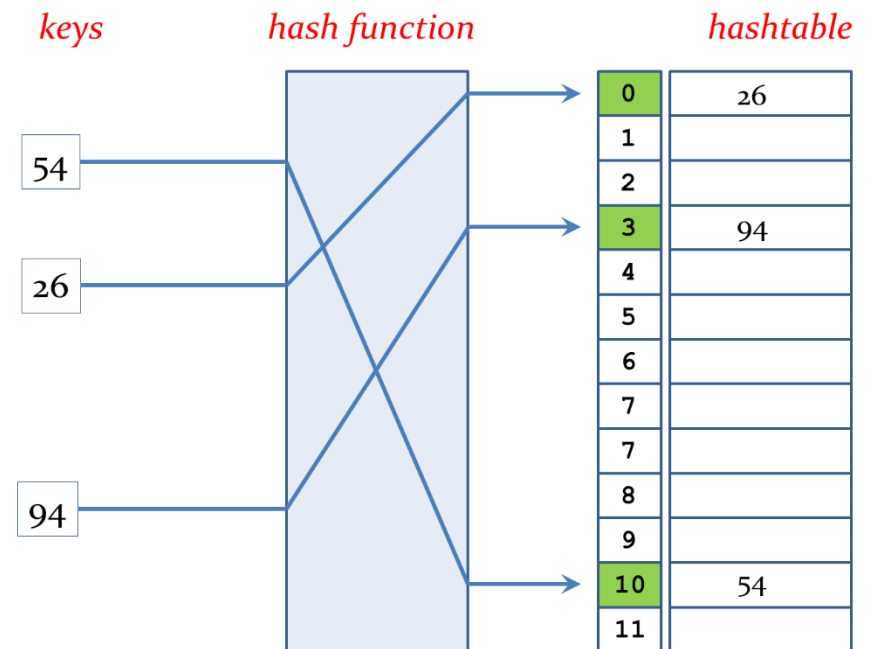
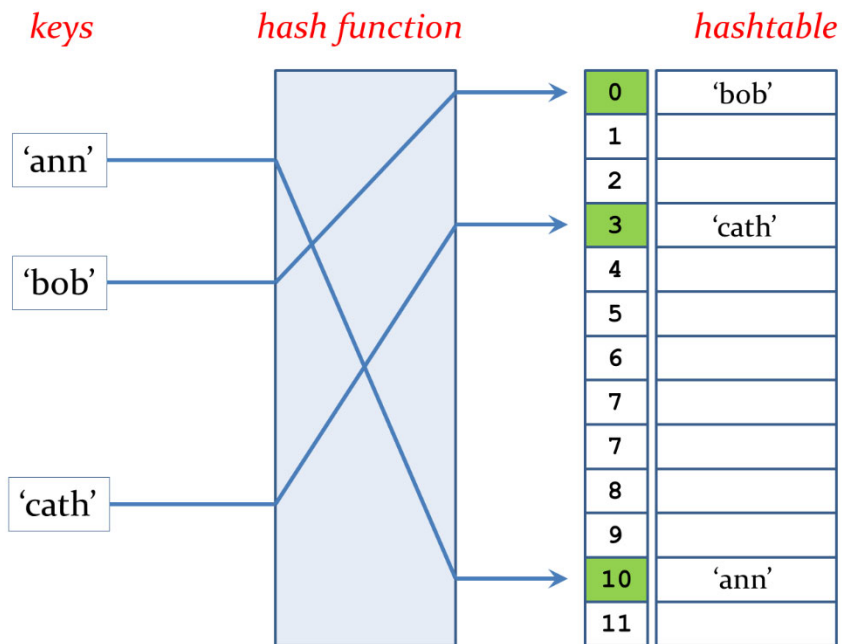
Collisions





Keys

- ▶ As we look at the basics of hashtables, we will disregard the *value* part of the *key/value* data, and just focus on the **keys**.
 - ▶ This means for simplicity we will use the key also as the value!
 - ▶ Keys will typically be either strings or integers:





Hash Functions

- ▶ The hash function maps the keys to index positions in the table
- ▶ Example:
 - ▶ Given integer keys and a table of size 13, we might use the following hash function, h :

$$h(key) = key \% 13$$

- ▶ Let's add the following keys: 54 26 94 17 77 31

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77

The **load factor**, λ , is the proportion of the hashtable that is full. In this case, $\lambda = 6 / 13 = 0.46$



Hash Functions - Mid-Square Method

- ▶ An alternative hash function for numeric keys is the mid-square method
- ▶ The key is squared, and some portion of the result is taken
- ▶ Advantage: consecutive keys are not mapped in consecutive table positions
- ▶ Example:
 - ▶ Square the key, take **all but the first** digit, modulo the table size

key	key ²	Remove first	% 13
655	529025	29025	9
654	427716	27716	0
653	426409	26409	6



Hash Functions - String Keys

- ▶ For keys which are strings, the ASCII values of the characters can be used to compute the index
- ▶ Example:
 - ▶ Add the ASCII values of each character in the key and take the sum modulus the table size

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL



Hash Functions - String Keys

- ▶ For keys which are strings, the ASCII values of the characters can be used to compute the index

- ▶ Example:

- ▶ Add the ASCII values of each character in the key and take the sum modulus the table size

	key	Add ord()s	Sum	% 13
>>> ord('a')				
97				
>>> ord('A')	"cat"	99 + 97 + 116	312	0
65				

```
def hash_string_1(s, tablesize):  
    sum = 0  
    for pos in range(len(s)):  
        sum = sum + ord(s[pos])  
    return sum % tablesize
```

```
print(hash_string_1('paul', 13))  
print(hash_string_1('does', 13))  
print(hash_string_1('dose', 13))  
print(hash_string_1('same', 13))  
print(hash_string_1('seam', 13))
```



5
11
11
6
6



Hash Functions - String Keys

- ▶ In the previous example, words that were anagrams hashed to the same index values
- ▶ We can avoid this problem by adding a “weight” to each character based on its position in the string (1 for the first, 2 for the second, etc.)

```
def hash_string_2(s, tablesize):  
    sum = 0  
    for pos in range(len(s)):  
        sum = sum + (pos+1) * ord(s[pos])  
    return sum % tablesize  
  
print(hash_string_2('paul', 13))  
print(hash_string_2('does', 13))  
print(hash_string_2('dose', 13))  
print(hash_string_2('same', 13))  
print(hash_string_2('seam', 13))
```



	10
	6
	5
	0
	4



Collisions

- ▶ Use the hash function

$$h(key) = key \% 13$$

- ▶ On the following hashtable

0	1	2	3	4	5	6	7	8	9	10	11	12
26	None	54	94	17	31	None	None	None	None	None	None	77

- ▶ What happens when we try to insert the key 44?

$$44 \% 13 = 5$$

- ▶ The keys 44 and 31 both map to index 5. We have a collision.



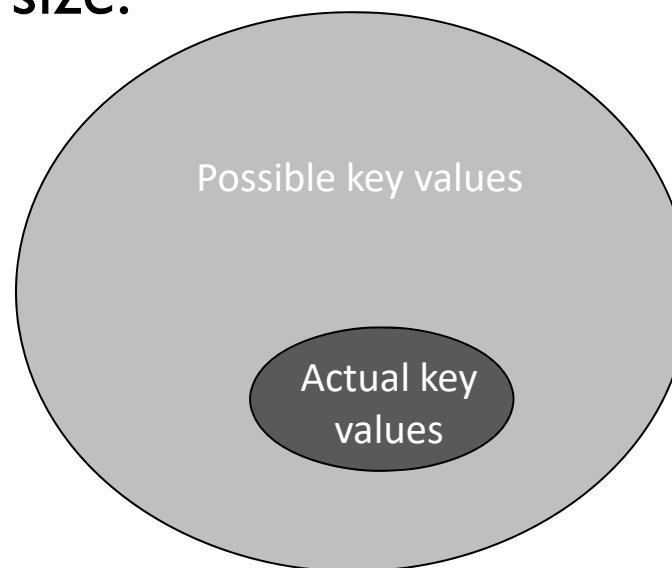
Perfect Hash Functions

- ▶ A hash function which **uniformly distributes** items over the whole hash table is a **perfect hash function**.
 - ▶ It is able to map m distinct items into a table of size m with **no collisions**
- ▶ One way to achieve this is to have a hash table which is big enough to accommodate the full range of keys. If the keys were eight digit student ID numbers we would need an 10^8 sized table. This may not be feasible.



Perfect Hash Functions

- ▶ This technique can be very wasteful as the number of students to be stored (and retrieved) may be much smaller than the actual table size.



We need some sort of compression from the full range of the keys into the number of hash table slots.



Good Hash Functions

- ▶ A good hash function should:
 - ▶ Be easy and fast to compute
 - ▶ Achieves even distribution of items (uniformity)
 - ▶ Ideally have a 1:1 correspondence between the number of items and the number of slots (i.e. size) of the hash table
- ▶ General requirements of a hash function:
 - ▶ The calculation of the hash function should involve the item value in its entirety
 - ▶ If a hash function uses modulo arithmetic, the base should be a **prime** number to help ensure even distribution of items



Exercise 1

- ▶ The hash function maps the keys to index positions in the table. Given integer keys and a table of size 13, we might use the following hash function, $h(\text{key}) = \text{key} \% 13$ – Let's add the following keys:
 - ▶ 19 28 64 91 73 42 89

0	1	2	3	4	5	6	7	8	9	10	11	12